



**TOBB ETU University of
Economics & Technology**

**Department of Computer
Engineering**

Senior Design Project

ANOMALY-BASED ACCIDENT DETECTION AND
FAULTY VEHICLE IDENTIFICATION SYSTEM (ADVIS)

Test Plan Report

Ahmet Faruk Sevim, Muhammet Emir Dizdar, Utku Çetin, Semih Uçan, Hasan Uzun

Supervisor: Assoc. Prof. Dr. Yücel Çimtay

Table of Contents

1. Introduction.....	3
2. Project Scope.....	3
2.1 Brief Description:	3
2.2 Main Functionalities of the System:.....	3
3. Features to Be Tested	4
3.1 Vehicle Detection	4
3.2 Multi-Object Tracking	4
3.3 Trajectory Extraction and Motion Analysis.....	4
3.4 Anomaly Detection in Traffic Behavior	4
3.5 Event Triggering and Video Clip Extraction	5
3.6 Accident Classification	5
3.7 Faulty Vehicle Identification.....	5
3.8 License Plate Recognition (LPR).....	5
3.9 Evidence Generation and Report Creation	5
3.10 Frontend User Interface	5
3.11 Backend Integration	5
3.12 AI Module Integration	6
4. Testing Methodology	6
4.1 Unit Testing	6
4.2 Integration Testing.....	6
4.3 System Testing.....	7
4.4 Performance Testing	7
4.5 User Testing.....	7
5. Test Environment.....	7
5.1 Hardware Environment	7
5.2 Software Environment.....	8
5.3 Testing Data.....	8
6. Test Cases	9
7. Risks and Limitations	11
7.1 Possible Technical Risks	11
7.2 Dataset and Hardware Limitations.....	12
8. Requirement Traceability Matrix	12

1. Introduction

This test plan describes the testing strategy for the project titled “Anomaly-Based Accident Detection and Faulty Vehicle Identification System (ADVIS)”. The purpose of this test plan is to verify that the developed system operates correctly, satisfies the specified project requirements, and produces accurate and reliable results. The testing process aims to evaluate the functionality of the system modules such as vehicle detection, anomaly detection, accident classification, and license plate recognition. In addition, the tests are designed to identify potential errors, validate system performance, and ensure the reliability of the overall traffic monitoring pipeline before final deployment.

2. Project Scope

Project Title: Anomaly-Based Accident Detection and Faulty Vehicle Identification System (ADVIS)

2.1 Brief Description:

The **Anomaly-Based Accident Detection and Faulty Vehicle Identification System (ADVIS)** is an intelligent traffic surveillance system designed to automatically detect traffic accidents and identify the faulty vehicle using computer vision and deep learning techniques. The system continuously monitors real-time video streams from traffic cameras and analyzes vehicle movements through object detection and multi-object tracking algorithms. By utilizing spatio-temporal anomaly detection, the system identifies irregular traffic behaviors such as collisions, sudden stops, or illegal maneuvers. When an anomaly is detected, the system triggers an automated diagnostic pipeline that extracts the relevant video segment and analyzes the event. The system then classifies the type of accident, determines the vehicle responsible for the incident based on trajectory analysis, and performs **License Plate Recognition (LPR)** to identify the faulty vehicle. Finally, the system generates a structured forensic report containing the accident type, vehicle identity, and supporting visual evidence, enabling authorities to review and investigate incidents efficiently.

2.2 Main Functionalities of the System:

The ADVIS system provides several key functionalities that enable automated traffic incident detection and analysis:

- **Real-time Vehicle Detection and Tracking**

The system detects vehicles in video streams using deep learning-based object detection models and assigns unique identifiers to track each vehicle’s trajectory

across frames.

- **Anomaly Detection in Traffic Behavior**

The system continuously analyzes vehicle trajectories and motion patterns to identify abnormal behaviors such as collisions, sudden deceleration, or irregular driving patterns.

- **Automatic Event Triggering and Video Extraction**

When an anomaly is detected, the system automatically extracts a short video segment containing the incident for further analysis.

- **Accident Classification**

The system analyzes the extracted video clip to classify the type of accident, such as rear-end collision or side impact.

- **Faulty Vehicle Identification**

Using trajectory analysis and rule-based evaluation of vehicle movements, the system determines which vehicle is responsible for the accident.

- **License Plate Recognition (LPR)**

The system detects and reads the license plate of the faulty vehicle using optical character recognition techniques.

- **Evidence Generation and Reporting**

The system generates a structured incident report containing accident type, timestamp, vehicle identification information, and supporting video evidence for further investigation.

3. Features to Be Tested

3.1 Vehicle Detection

The system must accurately detect vehicles from real-time video streams using deep learning-based object detection algorithms. The testing process will verify that vehicles such as cars, trucks, and motorcycles are correctly detected within the camera frame and represented with proper bounding boxes.

3.2 Multi-Object Tracking

The system must track detected vehicles across consecutive video frames and assign a unique identifier (ID) to each vehicle. Tests will verify that the tracking mechanism maintains temporal consistency and correctly follows vehicles even when they move across different regions of the frame.

3.3 Trajectory Extraction and Motion Analysis

The system must generate trajectory data for each detected vehicle, including position, movement direction, and velocity vectors. The tests will evaluate whether the system correctly records and processes vehicle motion data required for anomaly detection.

3.4 Anomaly Detection in Traffic Behavior

The anomaly detection module must analyze vehicle trajectories and identify abnormal traffic behaviors such as collisions, sudden stops, or irregular movements. The tests will verify that the system correctly distinguishes between normal traffic flow and anomalous events.

3.5 Event Triggering and Video Clip Extraction

When an anomaly is detected, the system must automatically trigger the diagnostic pipeline and extract the relevant video segment surrounding the detected incident. The testing process will ensure that the correct time window of the incident is captured and stored for further analysis.

3.6 Accident Classification

The system must analyze the extracted video clip and classify the detected incident into predefined categories such as rear-end collision or side-impact accident. Tests will verify that the classification module produces correct and consistent accident type predictions.

3.7 Faulty Vehicle Identification

The system must determine which vehicle is responsible for the detected accident by analyzing trajectory interactions and motion patterns before the collision. The testing process will evaluate whether the system correctly identifies the primary faulty vehicle.

3.8 License Plate Recognition (LPR)

The system must detect and read the license plate of the faulty vehicle using optical character recognition techniques. Tests will verify the accuracy of plate detection and character extraction under different visual conditions.

3.9 Evidence Generation and Report Creation

The system must generate a structured incident report containing accident type, timestamp, vehicle identification, and supporting visual evidence. The tests will verify that the generated reports contain accurate and complete information.

3.10 Frontend User Interface

The system must provide an interactive, web-based dashboard for real-time traffic monitoring and forensic analysis. Tests will verify role-based access control, the real-time rendering of live tracking visualizations, alert feed synchronization, incident archive searchability, and proper rendering of the forensic report modal.

3.11 Backend Integration

The backend system must correctly receive anomaly trigger requests from the AI module, persist incident and vehicle data, and broadcast real-time alerts via WebSocket. The retry mechanism must ensure reliable delivery even when temporary connection failures occur. Camera heartbeat signals must be processed to accurately reflect each camera's operational status.

User authentication, registration, and role-based access control must be verified to ensure that only authorized users can access protected resources.

3.12 AI Module Integration

The AI module must continuously ingest live video streams from RTSP cameras and maintain a rolling frame buffer to guarantee pre-event footage availability. Upon anomaly detection, the system must generate a valid video evidence clip from the buffer and deliver it to the backend. The RTSP ingestion pipeline must handle stream interruptions gracefully by automatically reconnecting without stopping the processing pipeline.

4. Testing Methodology

To ensure the correctness, reliability, and performance of the Anomaly-Based Accident Detection and Faulty Vehicle Identification System (ADVIS), multiple testing strategies will be applied throughout the development process. These testing approaches are designed to evaluate the system at different levels, from individual components to the complete integrated system.

4.1 Unit Testing

Unit testing focuses on verifying the correctness of individual modules and functions within the system. Each core component of the pipeline is tested independently to ensure that it performs its intended functionality without errors.

Examples of unit tests include testing the vehicle detection module to verify that vehicles are correctly detected from input frames, testing the tracking module to ensure that unique identifiers are assigned and maintained across frames, and validating the license plate recognition module to confirm that plate characters are accurately extracted from vehicle images. These tests help identify issues at the component level before integration with other modules.

4.2 Integration Testing

Integration testing evaluates the interaction between multiple modules within the ADVIS pipeline. Since the system consists of several interconnected subsystems such as detection, tracking, anomaly analysis, classification, and license plate recognition it is essential to ensure that data flows correctly between these modules.

For example, integration tests verify that the output of the vehicle detection module is correctly passed to the tracking module, that trajectory data generated by the tracking module is correctly interpreted by the anomaly detection module, and that triggered events correctly activate the diagnostic components such as accident classification and license plate recognition.

4.3 System Testing

System testing evaluates the complete integrated system to ensure that all components operate together as expected. In this stage, the entire ADVIS pipeline is tested using real or simulated traffic video streams.

The system is monitored to verify that it can detect vehicles, track their movements, identify anomalies, classify accidents, determine the faulty vehicle, and generate incident reports successfully. System testing ensures that the system meets the functional requirements defined for the project and that the end-to-end workflow operates correctly.

4.4 Performance Testing

Performance testing evaluates the efficiency and responsiveness of the system under realistic operating conditions. Since ADVIS is designed to process real-time video streams, it is important to measure how quickly and accurately the system can detect and analyze traffic incidents.

Performance tests analyze factors such as processing latency, detection accuracy, anomaly detection reliability, and computational resource usage (CPU/GPU utilization). The system is tested with different traffic scenarios and video conditions to ensure that it maintains acceptable performance levels without significant delays.

4.5 User Testing

User testing evaluates the usability and effectiveness of the system from the perspective of potential end users such as traffic operators or system administrators. In this stage, sample scenarios are used to simulate real-world accident situations.

Users review the generated accident reports and system alerts to verify that the information provided by the system is clear, accurate, and useful for decision-making. Feedback obtained during user testing helps identify usability improvements and ensures that the system meets practical operational needs.

5. Test Environment

The testing of the Anomaly-Based Accident Detection and Faulty Vehicle Identification System (ADVIS) will be conducted in a controlled environment that includes both the required hardware infrastructure and the software tools necessary to execute the system pipeline. The test environment is designed to simulate realistic traffic monitoring conditions while allowing accurate evaluation of the system's performance.

5.1 Hardware Environment

The system is tested on a high-performance computing environment capable of handling real-time video processing and deep learning inference tasks. The following hardware components are used:

- **Computer System:** A high-performance workstation or laptop capable of running deep learning models.
- **CPU:** Multi-core processor to handle data preprocessing, system control, and pipeline management.
- **GPU:** A CUDA-enabled GPU used for accelerating deep learning models such as object detection, anomaly detection, and video classification.
- **Memory (RAM):** Sufficient memory to process high-resolution video streams and maintain temporary buffers for video frames.
- **Storage:** High-speed SSD storage used for storing video clips, model files, and generated evidence data.
- **Camera / Video Input:** Traffic video datasets or recorded surveillance footage used as input for testing the system pipeline.

5.2 Software Environment

The ADVIS system is implemented using modern computer vision and deep learning frameworks. The following software tools and libraries are used during testing:

- **Operating System:** Linux-based operating system (Ubuntu) or Windows environment used for development and testing.
- **Programming Language:** Python, which is used to implement the computer vision and deep learning modules.
- **Deep Learning Frameworks:** PyTorch or TensorFlow for implementing detection, classification, and anomaly detection models.
- **Computer Vision Library:** OpenCV for image processing, frame extraction, and video stream handling.
- **Object Detection Model:** YOLO-based deep learning model used for vehicle detection.
- **Multi-Object Tracking Algorithm:** ByteTrack algorithm used for maintaining consistent vehicle tracking across frames.
- **OCR Engine:** Optical Character Recognition tools used for license plate recognition.
- **Database System:** SQLite or PostgreSQL used for storing incident metadata and generated reports.

5.3 Testing Data

To evaluate the system under different traffic scenarios, the following input data sources are used:

- Pre-recorded traffic surveillance videos
- Public traffic accident datasets
- Sample traffic scenarios representing different accident types

These datasets allow the system to be tested under various environmental conditions such as different lighting levels, traffic densities, and accident scenarios.

6. Test Cases

Test ID	Description	Input(s)	Expected Output	Result
TC-01	Vehicle detection test	Traffic video frame containing vehicles	Vehicles detected with correct bounding boxes	Passed
TC-02	Vehicle detection without vehicles	Empty road frame	No vehicle detected	Passed
TC-03	Multi-object tracking test	Video with multiple moving vehicles	Each vehicle assigned a unique ID and tracked correctly	Passed
TC-04	Tracking consistency test	Continuous video frames	Vehicle IDs remain consistent across frames	Partially Passed
TC-05	Trajectory extraction test	Vehicle tracking data	Correct trajectory coordinates generated	Passed
TC-06	Normal traffic behavior test	Video showing normal traffic flow	No anomaly detected	Passed
TC-07	Collision anomaly detection test	Video containing a vehicle collision	System detects anomaly and triggers alert	Partially Passed
TC-08	Sudden stop anomaly test	Vehicle suddenly braking	Anomaly score generated correctly	Partially Passed
TC-09	Event trigger test	Detected anomaly scenario	Diagnostic pipeline activated	Passed
TC-10	Video clip extraction test	Triggered anomaly event	Correct incident video clip extracted	Passed
TC-11	Accident classification test (rear-end)	Video showing rear-end collision	System classifies accident as rear-end collision	In Progress
TC-12	Accident classification test (side impact)	Video showing side impact collision	System classifies accident as side impact	In Progress
TC-13	Faulty vehicle identification test	Collision scenario video	Responsible vehicle correctly identified	In Progress
TC-14	License plate detection test	Vehicle image with visible license plate	License plate detected correctly	Passed
TC-15	License plate recognition test	Plate image	Correct alphanumeric plate extracted	Partially Passed
TC-16	Low visibility detection test	Night or rainy traffic video	Vehicles detected with acceptable accuracy	Partially Passed

TC-17	High traffic scenario test	Dense traffic video	System maintains stable tracking and detection	Passed
TC-18	Evidence generation test	Completed accident analysis	Evidence package generated successfully	In Progress
TC-19	Database storage test	Incident report data	Report correctly stored in database	Passed
TC-20	Full pipeline system test	Traffic video containing an accident	System detects accident, identifies faulty vehicle, reads plate, and generates report	In Progress
Tc-21	Frontend: Role-Based Access Control	User Credentials and Role Selection	Dashboard access granted or denied based on selected role	Passed
TC-22	Frontend: Live Perception View	Trajectory JSON Stream	Bounding boxes, tracklets, and anomaly charts render correctly	In Progress
TC-23	Frontend: Real-time Alert Feed	Anomaly Event Data Packet	Alert item appears in feed immediately with correct severity	In Progress
TC-24	Frontend: Incident Archives Table	API Response for Incidents	Incident records and historical data display properly in the table	In Progress
TC-25	Frontend: Forensic - Overview & Identity	Selected Alert Metadata	Modal shows AI collision summary and License Plate (OCR) data accurately	In Progress
TC-26	Frontend: Forensic - Fault Analysis & Evidence	Alert Forensic Parameters	System identifies the Primary Actor and displays the incident evidence correctly	In Progress
TC-27	Backend incident trigger processing test	AI module anomaly detection data	Incident entry created and stored in the database	Passed
TC-28	Backend trigger retry mechanism test	Backend unavailable during anomaly trigger	AI module retries and delivers incident data once connection is restored	Passed
TC-29	Camera heartbeat processing test	Periodic camera heartbeat signal	Camera status updated correctly in the system	In Progress
TC-30	RTSP video ingestion test	Live RTSP camera stream	Video frames captured continuously and stored in the buffer	Partially Passed
TC-31	RTSP stream reconnection test	Temporary RTSP stream interruption	System detects interruption and reconnects automatically	Partially Passed
TC-32	Circular buffer integrity test	Continuous video frame input	Buffer maintains correct frame count and replaces older frames	In Progress

TC-33	Video evidence generation test	Anomaly event and buffered frames	Valid video clip generated around the detected event	In Progress
TC-34	User registration test	Valid user registration information	New user account created and stored in the database	Passed
TC-35	Duplicate email registration test	Registration with existing email address	Request rejected, duplicate user creation prevented	Passed
TC-36	User login test	Valid user login credentials	User authenticated and access granted	Passed
TC-37	Invalid login credentials test	Login attempt with incorrect password	Login rejected, access denied	Passed
TC-38	Role-based access control test	Restricted operation by unauthorized user	Action blocked and unauthorized access prevented	Passed

7. Risks and Limitations

Although the Anomaly-Based Accident Detection and Faulty Vehicle Identification System (ADVIS) is designed to provide automated traffic accident detection and analysis, several technical risks and limitations may affect the system's performance during development and deployment.

7.1 Possible Technical Risks

- **Detection Errors:** Deep learning-based object detection models may occasionally fail to detect vehicles correctly or may produce false detections, especially in complex traffic scenes.
- **Tracking Instability:** Multi-object tracking algorithms may lose track of vehicles when occlusions occur, such as when vehicles overlap or temporarily leave the camera's field of view.
- **Anomaly Detection Accuracy:** The anomaly detection module relies on motion patterns and trajectory analysis. In some situations, unusual but non-dangerous driving behavior may be interpreted as an anomaly, leading to false alarms.
- **Accident Classification Limitations:** Video classification models may struggle to accurately categorize certain accident types if the collision occurs partially outside the camera view or if the event duration is extremely short.

- **License Plate Recognition Errors:** The license plate recognition module may produce incorrect results when license plates are partially occluded, blurred, or captured at unfavorable angles.

7.2 Dataset and Hardware Limitations

- **Limited Training Data:**
The performance of deep learning models strongly depends on the quality and diversity of the training dataset. A limited number of accident scenarios in the dataset may reduce model generalization.
- **Environmental Conditions:**
Lighting conditions such as night scenes, rain, fog, or strong reflections may reduce the visibility of vehicles and license plates, affecting detection and recognition accuracy.
- **Camera Angle and Resolution:**
The effectiveness of the system depends on the quality and positioning of the traffic cameras. Low-resolution cameras or poor camera angles may reduce detection accuracy.
- **Computational Resource Constraints:**
Since the system processes video data using multiple deep learning models, insufficient GPU resources may lead to slower processing speeds or delays in real-time analysis.

Despite these risks and limitations, the system architecture is designed to minimize their impact through modular design, anomaly-based triggering mechanisms, and continuous model improvement during development.

8. Requirement Traceability Matrix

The Requirement Traceability Matrix (RTM) establishes a direct relationship between the functional requirements of the Anomaly-Based Accident Detection and Faulty Vehicle Identification System (ADVIS) and the corresponding test cases defined in this test plan. The purpose of this matrix is to ensure that every functional requirement of the system is properly verified during the testing process.

By mapping requirements to their associated test cases, the RTM helps confirm that all major system functionalities including vehicle detection, anomaly detection, accident classification, license plate recognition, and report generation—are adequately tested. This traceability improves the transparency, completeness, and reliability of the testing process while ensuring that no critical requirement remains untested.

Requirement ID	Requirement Description	Related Test Case(s)
FR-01	The system shall detect vehicles in traffic video streams	TC-01, TC-02
FR-02	The system shall track multiple vehicles and assign consistent unique IDs across frames.	TC-03, TC-04
FR-03	The system shall generate trajectory data and motion information for detected vehicles.	TC-05
FR-04	The system shall detect abnormal traffic behavior such as collisions or irregular vehicle movements.	TC-06, TC- 07, TC-08
FR-05	The system shall trigger the diagnostic analysis pipeline when an anomaly is detected.	TC-09, TC-27, TC-28
FR-06	The system shall extract the relevant video segment related to the detected incident.	TC-10, TC-33
FR-07	The system shall classify the type of detected accident using video analysis.	TC-11, TC-12
FR-08	The system shall identify the vehicle responsible for the accident.	TC-13
FR-09	The system shall detect and read the license plate of the faulty vehicle.	TC-14, TC-15
FR-10	The system shall generate a structured accident report and store it in the system database.	TC-18, TC-19, TC-20
FR-11	The system shall provide a web-based dashboard for real-time monitoring and forensic review.	TC-21, TC-22, TC-23, TC-24, TC-25, TC-26
FR-12	The system shall ingest live video streams from RTSP cameras and maintain a continuous frame buffer for pre-event evidence capture.	TC-29, TC-30, TC-31, TC-32
FR-13	The system shall support user authentication, registration, and role-based access control to protect sensitive incident data.	TC-34, TC-35, TC-36, TC-37, TC-38

9. Conclusion

This test plan presents a comprehensive and systematic framework for evaluating the functionality, performance, and reliability of the **Anomaly-Based Accident Detection and Faulty Vehicle Identification System (ADVIS)**. By defining structured testing procedures and clearly specified test cases, the plan ensures that the system can be validated against its functional and non-functional requirements throughout the development lifecycle.

The testing methodology incorporates multiple levels of evaluation, including **unit testing, integration testing, system testing, performance testing, and user-oriented testing**. These approaches allow the individual components of the system—such as vehicle detection, multi-object tracking, anomaly detection, accident classification, license plate recognition, and evidence generation to be assessed both independently and as part of the complete integrated pipeline. Through these testing strategies, the system’s ability to process traffic video streams, detect anomalous events, identify responsible vehicles, and generate structured incident reports can be thoroughly verified.

The preliminary testing results demonstrate that several core modules of the system, including vehicle detection, tracking, and data storage, are already functioning reliably. However, some advanced components particularly those related to accident classification and fault attribution are still under active development and require further optimization and validation. Continuous testing and iterative improvements will be applied to enhance model accuracy, reduce false detections, and improve the robustness of the overall system under different environmental and traffic conditions.

Overall, the implementation of this test plan contributes significantly to improving the quality and reliability of the ADVIS system. By systematically identifying potential issues and validating system performance during development, the project can progress toward a stable and dependable solution for automated traffic accident detection and forensic analysis. With further refinement, expanded datasets, and additional testing, the system is expected to achieve a level of performance suitable for real-world deployment in intelligent traffic monitoring infrastructures.